1a) Explain the basic structure of a computer with a neat block diagram and describe the function of each unit.

Ans: Basic structure of a computer

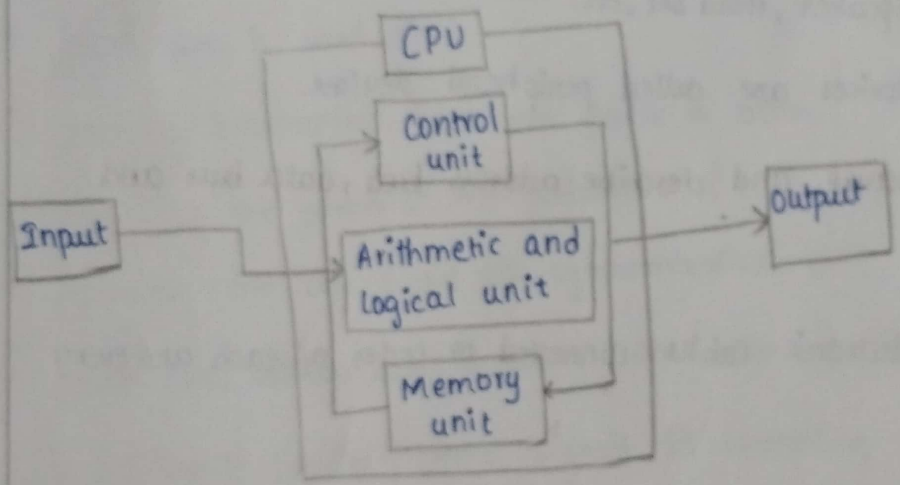
COMPUTER :

Commonly operating Machine Purposly used for technical and educational research.

→ Two types of computer

1. Analog computer
2. Digital computer

Functional units | Functional Block of computer :-



Input unit :- used to give input data using input devices

Ex:- Keyboard, mouse, scanner etc.

CPU :- Heart of the system (Central Processing Unit)

- It contains processor & controller

→ Control unit :- used to control sequence of operation.

→ ALU :- Used to perform arithmatic logical & shift operations

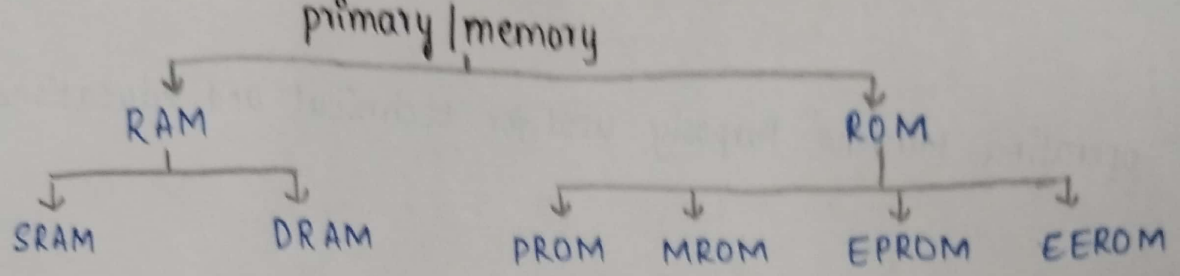
Ex :- +, -, x, /, Increment, decrement → Arithmatic (AND, OR, NOT) → logical

Scanned with OKEN Scanner

Shift operations :- Used to shift the data

Ex :- >> (SHR), << (SHL), ASHR, ASHL, ROR, ROL

Memory :- used to store the intermediate / final results



- Auxilary memory, Associative memory, cache memory, virtual memory are the types of memories

→ output units :-

used for displaying the results.

Ex :- Monitor, Printer, Speaker, Head set, etc.

- Input and output devices are called peripheral devices.

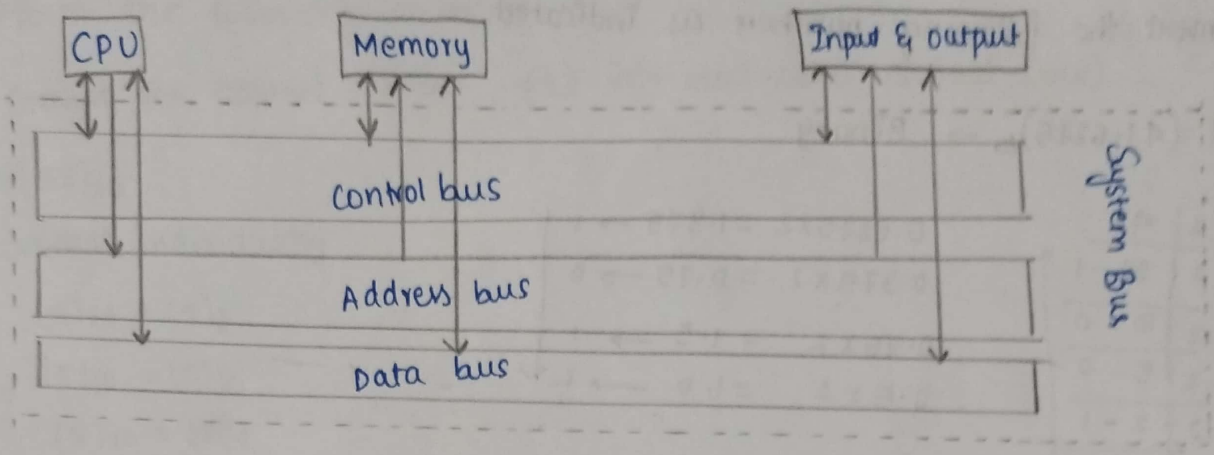
Explain system bus structure and describe address bus, data bus and control bus.

System bus :- It is electronic cables connected to codes of each and every device

- It is divided into 3 parts.

1. Address Bus
2. Data Bus
3. Control Bus

Scanned with OKEN Scanner



System bus :-

Single computer bus that connects the major components of computer system combining the functions of data bus to carry data, Address Bus to determine where it should be sent or receive from control Bus for controlling sequence of operations."

* System bus is used for,

1. Carrying information from one device to other.
2. Supplying the power to the peripherals.
3. providing the clk signal for synchronization.

- Control bus, Address bus, Data bus are called functions of System bus.

1. Control Bus :- Carries control signals for controlling and co-ordinating various activities in this system.
   - It is under controlled unit.
2. Address Bus :- Carries control memory address within the system.
3. Data Bus :- Transfer of data from one location to other within the system
   - Data can be image, Audio, Video, Text.

Scanned with OKEN Scanner

2a) Convert the following numbers as indicated

(i) $(41.6875)_{10}$ → Binary

2 | 41	
2 | 20 - 1	
2 | 10 - 0	
2 | 5 - 0	
2 | 2 - 1	
1 - 0	

$0.6875 \times 2 = 1.375 \rightarrow 1$

$0.375 \times 2 = 0.75 \rightarrow 0$

$0.75 \times 2 = 1.5 \rightarrow 1$

$0.5 \times 2 = 1.0 \rightarrow 1$

$\Rightarrow .1011$

$= 101001$

$(41.6875)_{10} = (101001.1011)_2$

(ii) $(101101.101)_2$ → decimal

$1\times2^5 + 0\times2^4 + 1\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0 . \; 1\times2^{-1} + 0\times2^{-2} + 1\times2^{-3}$

$= 32 + 8 + 4 + 1 . \; 0.5 + 0 + 0.125$

$= (45.625)_{10}$

$(101101.101)_2 = (45.625)_{10}$

(iii) $(736.4)_8$ → decimal

$7\times8^2 + 3\times8^1 + 6\times8^0 + 4\times8^{-1}$

$= 448 + 24 + 6 + 0.5$

$= (478.5)_{10}$

$(736.4)_8 = (478.5)_{10}$

(iv) $(7A3.F)_{16}$ → Binary

7 → 0111

A → 1010

3 → 0011

F → 1111

$(7A3.F)_{16} = (011110100011.1111)_2$

Scanned with OKEN Scanner

perform the following conversions.

(i) Convert the decimal number 458 into octal-coded decimal (OCD)

$(458)_{10}$

Convert individually

$(4)_{10} = (4)_8$

$(5)_{10} = (5)_8$

$(8)_{10} = (10)_8$

$OCD = (45\ 10)_8$

(ii) Convert the decimal number 789 into Hexa-coded decimal (HCD)

$(789)_{10} = (\ )_{16}$

$(7)_{10} = (7)_{16}$

$(8)_{10} = (8)_{16}$

$(9)_{10} = (9)_{16}$

$HCD = 789.$

(iii) Convert the decimal number 259 into binary coded decimal (BCD)

$(259)_{10} = (?)_2$

$(2)_{10} = 0010$

$(5)_{10} = 0101$

$(9)_{10} = 1001$

$BCD \Rightarrow 0010\ 0101\ 1001$

3 a) using 2's complement method, perform the following subtractions:-

(i) 1010100 - 1000011

2's complement of 1000011 :-

1's comp :- 0111100

+1

0111101

Add 1010100 + 2's comp of 1000011

```
  1 0 1 0 1 0 0
+ 0 1 1 1 1 0 1
---------------
(1) 0 0 1 0 0 0 1
```

↳ carry discarded

Answer is 0010001

(ii) 100101 - 110011

2's comp of 110011

1's 001100

+1

001101

Now add 100101 + 2's comp of 110011

```
  1 0 0 1 0 1
  0 0 1 1 0 1
-------------
  1 1 0 0 1 0
```

∴ No carry is generated, result will be negetive of that 2's comp

result :- -[001110].

b) Represent the following in fixed-point signed magnitude and 2's complement (8-bit):

(i) 25

Binary of 25 :- 00011001

MSB is equal to 0 for positive sign.

2's complement of positive numbers remain same ⇒ 00011001

(ii) -25

Binary of 25 :- 00011001

Sign magnitude

Sign bit = 1

Magnitude = 0011001

Scanned with OKEN Scanner

= 10011001

2's comp :-

1's :- 11100110
+ 1
11100111

⇒ 11100111

4a) Explain floating point representation and normalization.

- floating point representation :-
- floating point representation is a method used to store real (decimal) numbers in a computer using scientific notation.
- Since computers cannot store large or fractional numbers directly, numbers are represented as Base = 2 (binary system)
- General form of floatting point number $= M \times 2^E$

Where,

M (mantissa / significand) → significant digits.

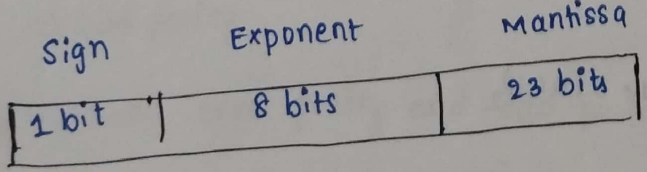
E (Exponent) → position of decimal point.

→ There are two types of representations :-

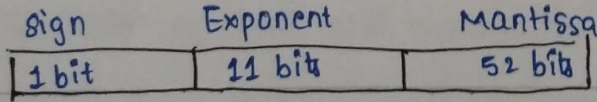
(i) Single precision

(ii) Double precision

(i) IEEE - 754 single precision format (32 bits)



(ii) IEEE - 754 double precision format (64 bits)



Scanned with OKEN Scanner

• Normalization :-

• Normalization is the process of adjusting the binary point so that only one non-zero digit appears before the decimal point.

• In binary, the digit before decimal must be 1.

Normalized form,

$1.xxxx.x \times 2^E$

Ex :- $(1100.1)_2$

1.1001×2^3

4b) Represent the following numbers in IEEE-754 single precision format

(i) 12.5

Given that,

$(12.5)_{10} = (1100.1)_2$

Normalization :-

Move decimal after first 1 :-

$1100.1 = 1.1001 \times 2^3$

$E = 3$

$E = 127 + 3 = 130$

$(130)_{10} = (10000010)_2$

Sign	Exponent	Mantissa
0	10000010	10010000000000000000000

(ii) 263.3

Given that,

$(263.3)_{10} = (100000111.01001)_2$

Normalization

$1.00000111010011 \times 2^8$

$E = 8$

Scanned with OKEN Scanner

$e = 127 + 8 = 135$

$(135)_{10} = (10000111)_2$

Sign	Exponent	Mantissa
0	10000111	00000111010011001100110

5a) Explain error detection using parity bits. distinguish between even parity and odd parity with suitable examples.

Sol Error detection codes :-

- Binary data transmitted through a communication medium may be affected by noise.
- Noise can change bit from 1 to 0 or 0 to 1
- An error detection code is used to detect errors during data transmission
- Detected errors cannot be corrected, but their presence can be identified.
- If an error is detected the data is usually transmitted.
- If errors occur too frequently, the system is checked for malfunction.

Parity Bit :-

- The parity bit is the most commonly used error detection method.
- A parity bit is an extra bit added to the binary message
- It makes the total number of 1's either even or odd

→ Two types of parity :-

a) Even parity

b) Odd parity.

- Distinguish between even parity and odd parity :-

→ Even parity :-

- The parity bit is chosen so that total number of 1's is even

Scanned with OKEN Scanner

- If the sum of 1's in the message is already even, paritybit = 0
- If the sum of 1's is odd, parity bit = 1

Example :-

Data = 1011

Number of 1's = 3(odd)

To make total even → parity bit = 1

Transmitted data = 10111

Total number of 1's = 4 (even)

Odd parity :-

- The parity bit is chosen so that the total numbers of 1's is odd.
- If the sum of 1's in the message is already odd, parity-bit = 0
- If the sum of 1's is Even, parity bit = 1.
- The odd parity bit is the complement of the even parity Bit.

Example :-

Data = 1010

Number of 1's = 2 (Even)

To make total odd → parity bit = 1

Transmitted data = 10101

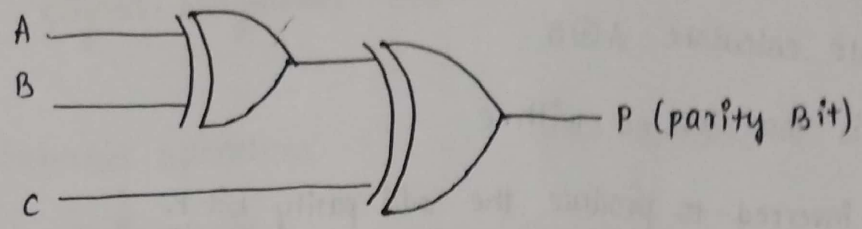
Total number of 1's = 3 (odd).

b) Draw the odd parity generator circuit and explain its operation.

An Odd parity Generator is a combinational logic circuit used to generate a parity bit such that the total number of 1's in the transmitted data (data bits + parity bit) is odd.

- It is mainly used in error detection during data transmission in digital compun communication systems

Block / Logic diagram (3-bit example)



Where A, B, C → Input data bits

P → generated odd parity bit

XOR gates are used to generate parity

Boolean Expression

For a 3-bit odd parity generator:

$$P = \overline{A \oplus B \oplus C} = (A \oplus B \oplus C)'$$

This ensures that the total number of 1's including the parity bit becomes odd.

Truth table :-

A	B	C	A⊕B⊕C	P (odd parity Bit)
0	0	0	0	1
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	0

→ After adding P, the total number of 1's becomes odd.

Scanned with OKEN Scanner

Operation :-

1. The XOR gate outputs 1 when the number of input 1's is odd
2. First XOR gate calculate $A \oplus B$
3. The output is then XORed with C
4. The result is inverted to produce the odd parity bit P.
5. The parity bit is transmitted along with the original data bits.
6. At the reciever side, a parity checker verifies whether the number of 1's is odd. if not, an error is detected.

Ex :- let the input data be

$A = 1$
$B = 0$
$C = 0$

$A \oplus B \oplus C = 1$

$P = 0$

Transmitted data : 1000

Number of 1's = 1 → odd parity satisfied

6. a) Draw a flow chart for signed-magnitude Addition and Subtraction and explain its operation

In the signed magnitude representation, the most significant bit (MSB) represents the sign :

- 0 → positive
- 1 → negative

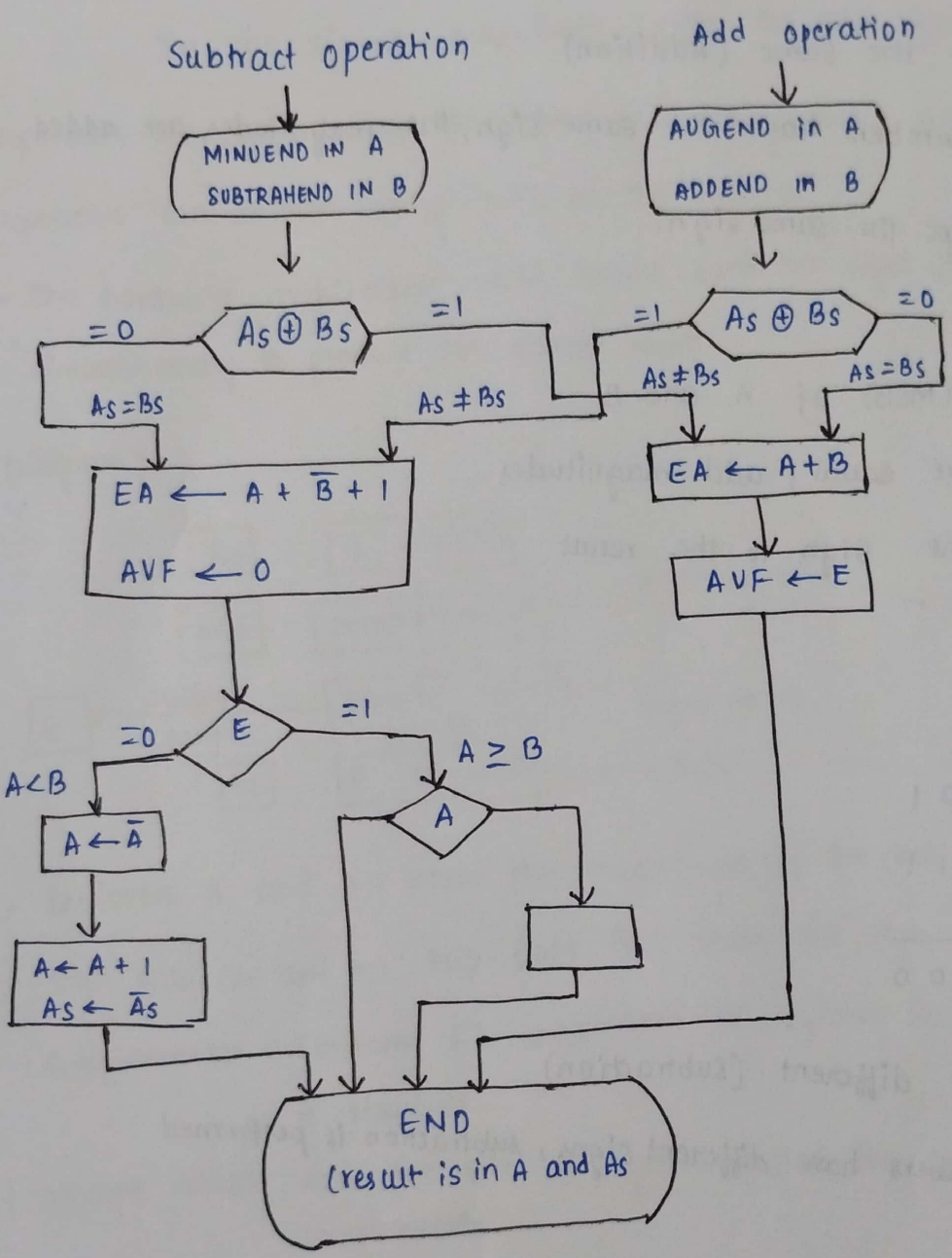
The remaining bits represent the magnitude (absolute value).

For example (5-bit system)

Number	Signed Magnitude
+6	00110
-6	10110

Scanned with OKEN Scanner

Flowchart of Addition and Subtraction with Signed-Magnitude Data



Operation of Signed Magnitude Addition

The operation depends on whether the signs of the two numbers are same or different

Case 1 : Signs are the same (Addition)

If both numbers have the same sign, the magnitudes are added and the result keeps the same sign.

Steps

1. check sign bits (MSB) of A and B
2. If both signs are equal, add magnitudes
3. Assign the same sign to the result

Example

+5 + +3

+5	0	0 1 0 1
+3	0	0 0 1 1
+		
+8	0	1 0 0 0

Case-2 : Signs are different (Subtraction)

If the numbers have different signs, subtraction is performed

Steps :

1. Compare magnitudes of A and B
2. Subtract the smalled magnitude from the larger magnitude
3. The result sign is the sign of the number with larger magnitude

Ex :- +7 + (-4)

+7 : 0 0 1 1 1 — Magnitude 7-4=3

-4 : 1 0 1 0 0 — sign = positive

Scanned with OKEN Scanner

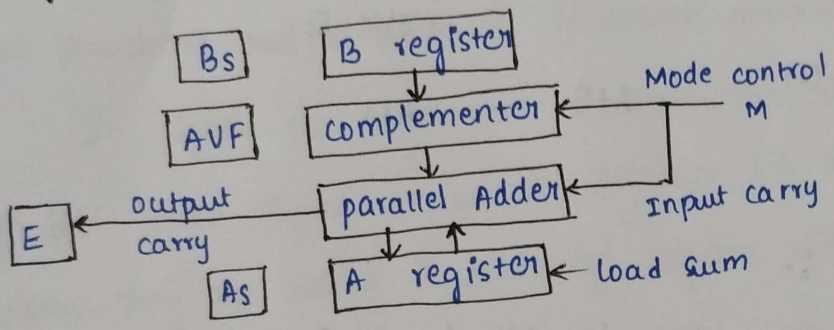
Explain the Hardware architecture of signed-magnitude addition and Subtraction

Sol Hardware Architecture for signed-magnitude Addition and Subtraction

6M In the signed-magnitude system, the most significant Bit (MSB) represents the sign. (0 for positive, 1 for negative), and remaining bits represents the absolute magnitude of the number.

- The hardware architecture must handle both the signs and the magnitudes simultaneously to produce the correct result.

Diagram :-



- Registers A and B : store the magnitude of the two numbers
- Sign Bits (As and Bs) : Flip-flops that store the sign of the numbers
- Complementer : Performs 1's complement of register B when subtraction is required.
- parallel Adder : Adds the bits of register A and the output of the complementer.

- Mode Control (M) :
  - If M = 0, the circuit performs addition
  - If M = 1, the circuit performs Subtraction
- Output carry (E) : A flip-flop that stores the carry-out from the most significant bit of the parallel adder.

Scanned with OKEN Scanner

- Add-overflow-Flip-flop (AVF): Indicates if the result of an addition exceeds the size of the register

Hardware logic table

operation	Sign condition (As ⊕ Bs)	Magnitude Action	Result Sign (As)
Add	same (0)	A + B	As
Add	Different (1)	A - B	if $A \geq B$ then As; else $\bar{A}_s$
Subtract	Same (0)	A - B	if $A \geq B$ then As; else $\bar{A}_s$
Subtract	different (1)	A + B	As.

Working procedure :-

1. Determine the operation. :-

The hardware first compares the sign bits As and Bs.

- If we are adding and signs are the same, OR if we are subtracting and signs are different → perform Magnitude addition
- Otherwise → perform Magnitude subtraction.

Execution of Magnitude Addition (A+B) :-

- The magnitudes in A and B are added by the parallel adder
- The carry-out is stored in flip-flop E.
- If E=1, the overflow flip-flop (AVF) is set to 1, indicating the result is too large for the register.

Scanned with OKEN Scanner

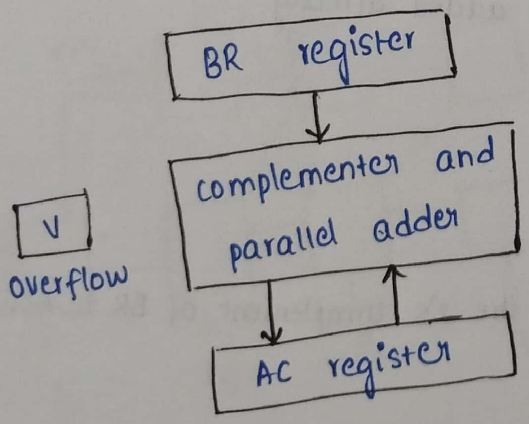
Execution of Magnitude subtraction (A-B):

- The parallel adder performs $A + \overline{B} + 1$.
- Condition A (Carry E=1): This means $A \geq B$ register A contains the correct result. If A=0, the sign As is set to 0 (to avoid -0)
- Condition B (carry E=0): This means $A < B$. The result in A is in 2's complement form the hardware must:
  - Take the 2's complement of A to get the true magnitude
  - Complement the sign bit As (i.e., $As = \overline{As}$).

7a) Explain the 2's complement addition and subtraction method with a neat flowchart and describe its hardware architecture

In signed 2's complement representation, the left most bit is the sign bit. Unlike signed-magnitude, the sign bits are not separated from the registers. They are added or subtracted together with the magnitude bits in the same parallel adder

Hardware architecture :- The hardware consists of registers and a parallel adder to process the numbers.



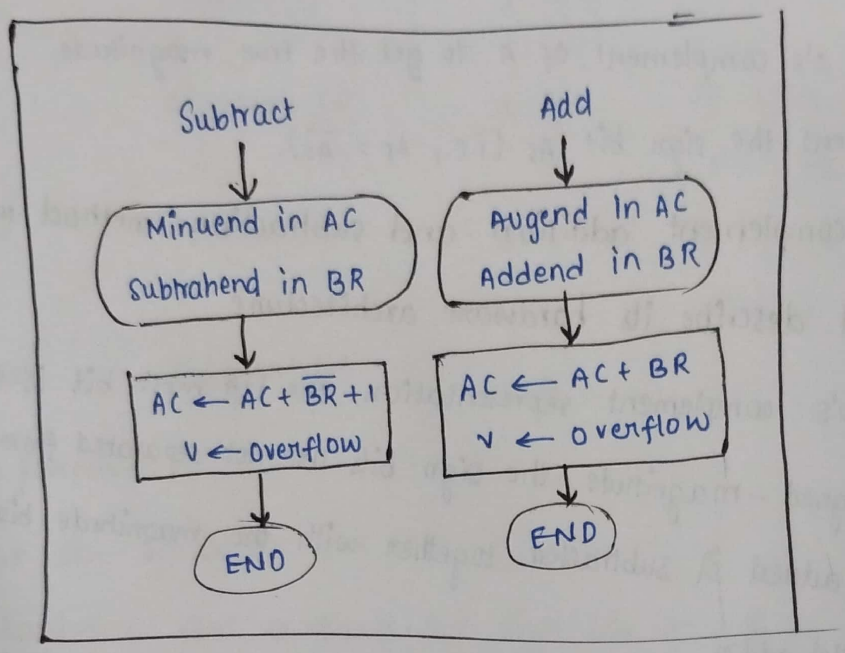
- AC (Accumulator) : It stores the first operand (Augend | Minuend).
- BR Registers : It stores the second operand (Addend | Subtrahend)

Scanned with OKEN Scanner

- Complementer and parallel adder : This units performs the actual addition. For subtraction, the complementer inverts the bits of BR.
- V (overflow Flip-flop) : This flip-flop detects if the result is too large for the register.

Explain neat

Flow chart :-



procedure :

1. Addition :
   - The content of AC and BR are added directly
     $AC \leftarrow AC + BR$
   - The overflow bit V is updated
2. Subtraction :
   - Subtraction is done by adding the 2's complement of BR to AC.
   - $AC \leftarrow AC + \overline{BR} + 1$
   - Here, $\overline{BR}$ is the 1's complement, and the +1 is provided by the input carry to adder.

Scanned with OKEN Scanner

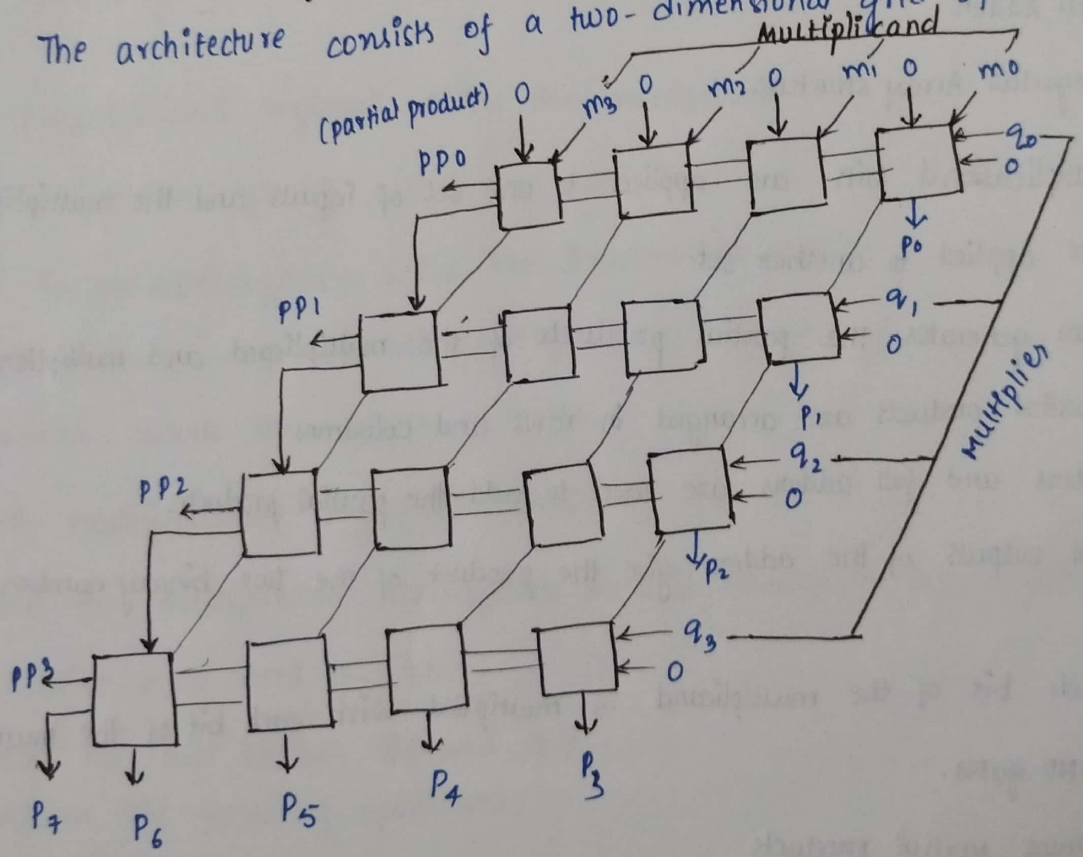
Explain the architecture and working of an Array Multiplier with neat diagram.

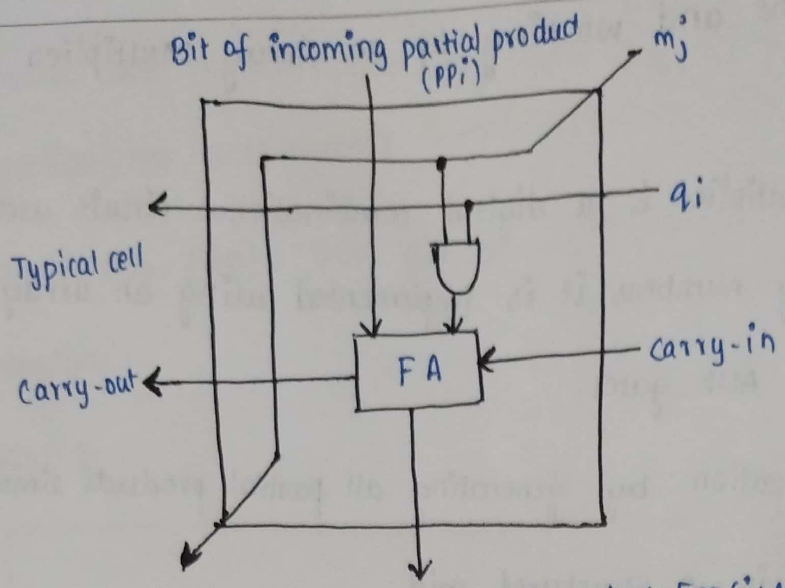
An array Multiplier is a digital combinational circuit used for multiplying two binary numbers. It is implemented using an array of Full adders (FA) and AND gates.

- It performs multiplication by generating all partial products simultaneously and then adding them in a structured grid.
- For an n×m bit multiplication, it requires n×m AND gates and a specific number of adders.

Architecture of Array Multiplier.

The architecture consists of a two-dimensional grid of "cells"





The architecture of an array multiplier consists of the following components

- AND gates
- Half Adders
- Full Adders
- Regular Array structure.

1. The multiplicand bits are applied to one set of inputs and the multiplier bits are applied to another set.
2. AND gates generates the partial products of the multiplicand and multiplier bits
3. These partial products are arranged in rows and columns
4. Half adders and full adders are used to add the partial products.
5. The final outputs of the adders give the product of the two binary numbers

Working :-

1. First, each bit of the multiplicand is multiplied with each bit of the multiplier using AND gates.
2. This produces partial products
3. The partial products are shifted according to their bit position
4. These shifted partial products are then added using half adders and full adders arranged in the array.

Scanned with OKEN Scanner

5. The carries generated during addition are propagated to the next stage.
6. Finally, the outputs of the last stage produce the final binary product

a) Explain Booth's multiplication algorithm with a neat flowchart and advantage

Booth's multiplication algorithm is an efficient method used for multiplying signed binary numbers in 2's complement representation.

- It reduces the number of addition and subtraction operations during multiplication. The algorithm examines the multiplier bits and performs addition, subtraction, or shift operations accordingly

Registers used in Booth Algorithm

The algorithm uses the following registers:

- M (Multiplicand register) – stores the multiplicand.
- Q (Multiplier register) – stores the multiplier.
- A (Accumulator register) – stores the intermediate results.
- Q1 – extra flip-flop used with Q register.
- Counter – counts the number of cycles.

Booth multiplication Algorithm (steps)

1. Load the multiplicand into register M and multiplier into register Q.
2. initialize A = 0 and Q-1 = 0
3. check the pair of bits Q0 and Q-1
4. Perform the following operations:
   - If Q0 Q-1 = 00 or 11 → No operation, perform Arithmetic Right shift
   - If Q0 Q-1 = 01 → A = A + M, then Arithmetic Right shift
   - If Q0 Q-1 = 10 → A = A - M, then Arithmetic Right shift

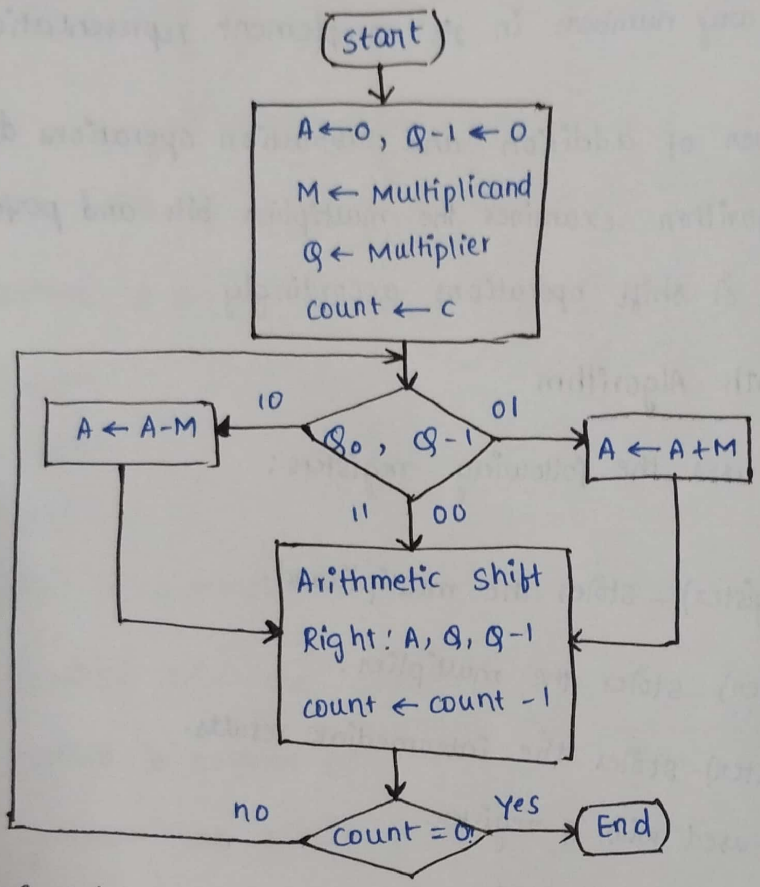
Scanned with OKEN Scanner

5. Decrement the counter.

6. Repeat the process until the counter becomes zero

7. The final product is stored in A and Q registers

4. Flowchart of Booth Algorithm



working Principle :-

The control logic checks the two bits Q0 and Q-1

- If both bits are equal, the algorithm performs only Arithmetic right shift
- If the bits are 01, the multiplicand is added to accumulator A.
- If the bits are 10, the multiplicand is subtracted from A

After each operation, the registers A, Q, and Q-1 are shifted right arithmetically to preserve the sign of the number. The final multiplication result appears in registers A and Q.

Scanned with OKEN Scanner

Advantages of Booth Algorithm

1. It reduces the number of addition operations during multiplication
2. It can efficiently handled signed numbers in 2's complement form.
3. It improves multiplication speed compared to basic-shift-add methods.
4. It is widely used in hardware multipliers in computer processors.

8b) Multiply the following using Booth Algorithm : (+6) x (-5)

1. Convert numbers into binary (4-bit 2's complement)

Multiplicand = M = +6

$(6)_{10} = (0\ 1\ 1\ 0)_2$

Multiplier Q = -5

$(+5)_{10} = (0\ 1\ 0\ 1)_2$

2's of -5 = 1011

So, M = 0110

-Q = 1010

Q = 1011

2. Initial registers

A = 0000

Q = 1011

Q-1 = 0

M = 0110

count = 4

Scanned with OKEN Scanner

Booth multiplication table

A	Q	Q-1	operation
0000	1011	0	initial values
1010	1011	0	A = A-M
1101	0101	1	Arithmetic Right shift
1110	1010	1	Arithmetic Right shift
0100	1010	1	A = A+M
0010	0101	0	Arithmetic Right shift
1100	0101	0	A = A-M
1110	0010	1	Arithmetic Right shift

4. combine A and Q

AQ = 1110 0010

5. Binary result = $(1110\,0010)_2$

6. Decimal result = $(+6) \times (-5) = -30$

∴ The product of (+6) and (-5) using Booth's Algorithm is -30.

9a) Draw the hardware architecture of Booth's multiplication algorithm and explain its operation.

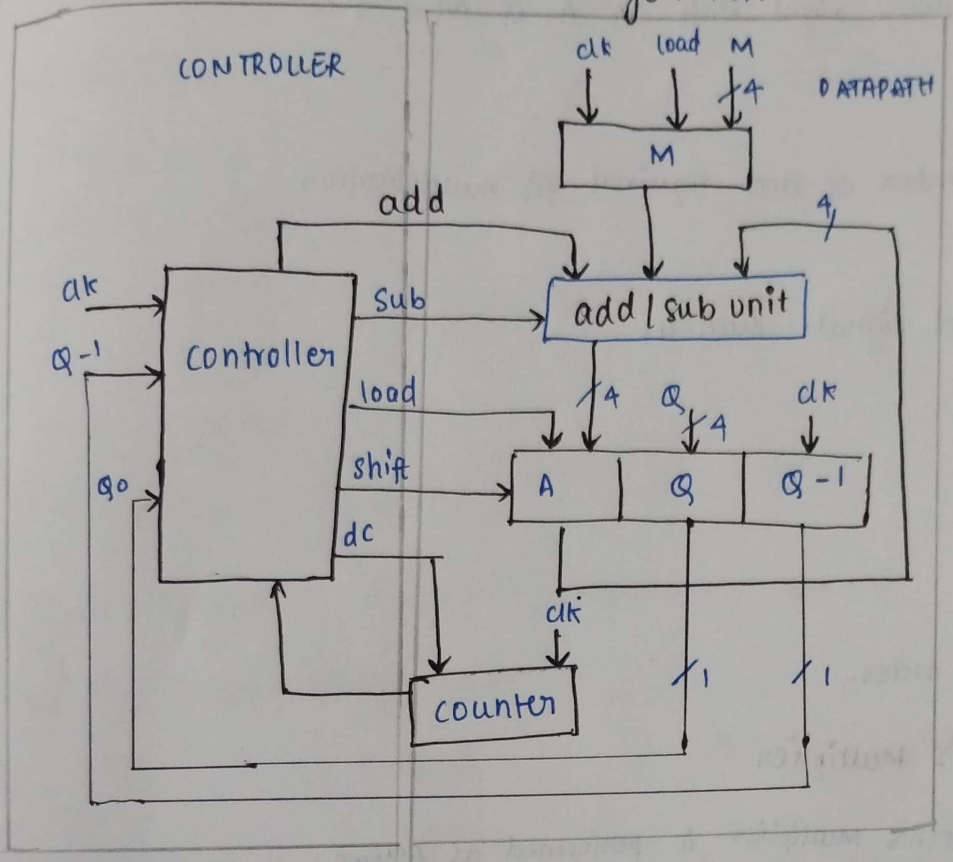
The hardware implementation of Booth's multiplication Algorithm consists of two main parts

- Controller
- Datapath.

The datapath contains the registers and arithmetic units required for multiplication. The controller generates control signals that control the operations

performed in datapath

Hardware for Booth Algorithm.



Main components :-

1. M register (Multiplicand Register)
   stores the multiplicand
2. Q Register (Multiplier Register)
   Stores the multiplier
3. A Register (Accumulator)
   Stores intermediate and partial results of multiplication
4. Q-1 Flip-flop
   stores the previous least significant bit of Q register
5. Adder / subtractor (ALU)
   performs ~~(arithemetic right)~~ addition or subtraction between A and M

6. Arithmetic Right shift unit

performs arithmetic right shift on A, Q and Q_{-1}.

7. Down counter

Counts the number of steps required for multiplication

8. Controller.

Generates control signals such as

- Load
- Add
- Subtract
- shift
- Decrement order.

Operation of Booth's Multiplier

The operation of Booth's Multiplier is performed as follows:

1. Load the multiplicand into M register and multiplier into Q register
2. initialize $A=0$ and $Q_{-1}=0$
3. set the counter equal to ~~(zero)~~ the number of bits in multiplier
4. Examine the pair of bits Q_0 and Q_{-1}
   - If $Q_0, Q_{-1} = 00$ or $11 \rightarrow$ perform arithemetic rightshift of A, Q and Q_{-1}.
   - If $Q_0 Q_{-1} = 01 \rightarrow$ Add multiplicand to accumulator ($A = A+M$).
   - If $Q_0 Q_{-1} = 10 \rightarrow$ Subtract multiplicand from accumulator ($A = A-M$).
5. store the result in A register.
6. Performs arithemetic right shift of A, Q and Q_{-1} to preserve the sign bit
7. decrement the counter by 1.
8. Repeat the process until the counter becomes zero
9. The final product is obtained from the combined contents of A & Q registers

Multiply the following using Booth Algorithm : (-4)*(+7)

1. Convert numbers to binary

Multiplicand M = -4

+4 = 0100

1's 4 = 1011

2's 4 = 1100

∴ M = 1100

-M = 0100

Multiplier Q = +7 = 0111

2. Initial values :- A = 0000 ; Q = 0111 Q-1 = 0

Booth multiplication Table

A	Q	Q-1	Operation
0000	0111	0	initial values
1100	0111	0	A = A - M
1110	0011	1	Arithmetic Right shift
1111	0001	1	Arithmetic Right shift
1111	1000	1	Arithmetic Right shift
0011	1000	1	A = A+M
0001	1100	0	Arithmetic right shift

Combine A and Q

AQ = 00011100 = 28

1's AQ = 11100011

2's AQ = $(11100100)_2 = -28$.

Scanned with OKEN Scanner

10a) Explain restoring division algorithm and describe its hardware architecture.

Restoring Divison Algorithm

The restoring divison algorithm is a method used in digital computers to perform binary divison. In this method, the divisor is subtracted from the partial remainder. If the result becomes negative, the original value is restored by adding the divisor back.

This algorithm is similar to the longhand divison method used in decimal arithmetic.

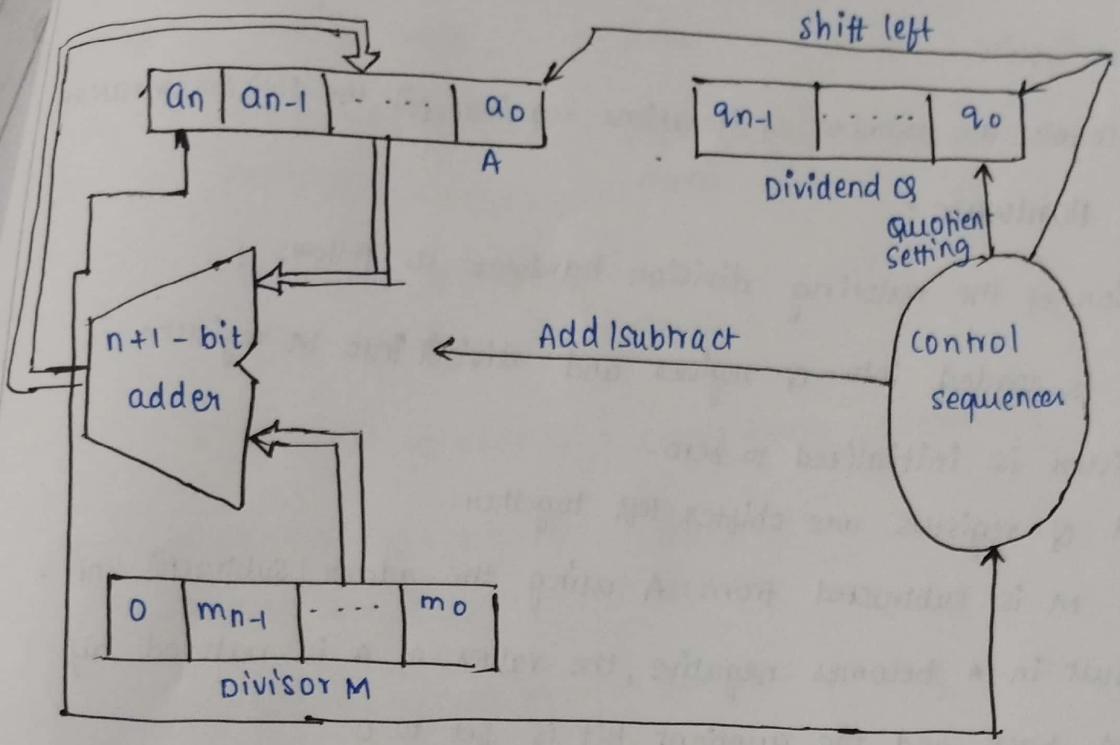
Algorithm steps :-

The restoring divison algorithm performs the following steps.

1. Load the divisor into register M and the dividend into register Q.
2. Initialize the accumulator register A to 0.
3. Repeat the following steps n times.
   a. Shift A and Q left by one bit position
   b. Subtract M from A and store the result in A.
   c. Check the sign of A:
   - If A is negative, set $Q_0 = 0$ and restore A by adding M back to A
   - If A is positive, set $Q_0 = 1$.
4. After completion of n steps :
   - Quotient is stored in Q register
   - Remainder is stored in A register

Scanned with OKEN Scanner

Hardware Architecture of Restoring division



Hardware for binary division

The hardware implementation of restoring division consists of several registers and arithmetic units.

Main components :-

- A register (Accumulator) : stores the partial remainder during the division process.
- Q Register (Dividend / Quotient ~~Remai~~ gister) : Initially stores the dividend and finally holds the quotient.
- M register (Divisor Register) : stores the divisor
- Adder/Subtractor unit : performs subtraction of the divisor from the accumulator.
- shift unit : performs left ~~uni~~ shift operation on A and Q registers.

Scanned with OKEN Scanner

- Control logic unit: controls the sequence of operations such as shift, subtract and restore.
- Counter: Counts the number of iterations required for the division operation

Operation of Hardware:

The operation of the restoring division hardware as follows:

1. The dividend is loaded into Q register and divisor into M registers
2. The A register is initialized to zero.
3. The A and Q registers are shifted left together.
4. The divisor M is subtracted from A using the adder/subtractor unit.
5. If the result in A becomes negative, the values of A is restored by adding M back and the quotient bit is set to 0.
6. If the result is positive, the quotient bit is set to 1
7. The process continues until the counter becomes zero.
8. Finally, the quotient is stored in Q register and remainder in A A register.

10b) perform restoring divison for: $101101 \div 101$

Dividend has 6 bits, so we perform 6 steps

Divisor

M = 00101

Initial values

A = 000000

Q = 101101

Restoring Division Table

Step	Shift A, Q	A-M	Result	Q_0
1.	A = 00001 Q = 011010 Restore A = 00001	00001 - 00101	Negative	0
2.	A = 00010 Q = 110100 Restore A = 00010	00010 - 00101	-ve	0
3.	A = 00101 Q = 101000	00101 - 00101	00000	1
4.	A = 00001 Q = 010010 Restore A = 00001	00001 - 00101	-ve	0
5.	A = 00010 Q = 100100 Restore A = 00010	00010 - 00101	-ve	0
6.	A = 00101 Q = 001000	00101 - 00101	00000	1

Final result :-

Quotient (Q) = 001001

Remainder (A) = 0000

45 ÷ 5 = 9

$(001001)_2 = 9$.

Scanned with OKEN Scanner